ACTIVELY HIRING

Build "Veloce": AI-Powered Project Intake & Estimation Engine

Complete this assessment to be considered for the **Full-Stack AI Engineer** role at CodeAcme. It tells us far more than a resume ever could.

2-3 focused days | AI-assisted workflows expected | All free-tier infrastructure

An agency receives project briefs from potential clients through a public intake form. An AI pipeline automatically analyzes each brief to extract structured requirements, estimate cost and timeline, and assign a complexity score. The agency team manages the entire pipeline from a real-time internal dashboard.

MODULE 01

Public Intake & AI Analysis Pipeline

Public Intake Form: A clean, accessible form where a prospective client submits: project title, description (rich text), budget range (dropdown tiers), timeline urgency, and contact info. Rate-limited (Upstash Redis) with proper input validation and error states.

AI Brief Analyzer: On submission, an async AI pipeline (any model/provider: OpenAI, Claude, OpenRouter, Gemini, Groq, free-tier credits, open-source models, whatever works) that:
(a) Extracts a structured list of features/requirements from the description
(b) Classifies the project category (Web App, Mobile, AI/ML, Automation, Integration)
(c) Generates an estimated effort range in hours and a suggested tech stack
(d) Assigns a complexity score (1-5)
Store the raw brief and AI analysis as separate linked records.

Webhook Endpoint: A POST `/api/webhooks/intake` endpoint that accepts a JSON brief payload with HMAC signature verification, simulating external sources (Typeform, Zapier, etc.) feeding briefs into the same pipeline. Must process identically to form submissions.

MODULE 02

Internal Dashboard & Pipeline Management

Auth & Roles: Email/password authentication. Two roles: Admin (full access, can assign briefs) and Reviewer (can view and add notes to assigned briefs only).

Pipeline Kanban: Briefs flow through stages: New → Under Review → Proposal Sent → Won → Archived. Drag-and-drop stage transitions with optimistic UI updates. Each stage change logs a timestamped

event.

Brief Detail View: Original submission and full AI analysis side-by-side, internal notes thread (threaded comments between team members), assignment history, and stage timeline. Reviewer can override/edit AI-generated estimates with a reason field.

Real-Time Updates: When a new brief is submitted or a stage changes, all connected dashboard users see updates without refreshing. Use Server-Sent Events, polling, or WebSockets.

Analytics Dashboard: Overview: briefs by stage (bar chart), average AI complexity score over time, conversion rate (Won / Total), estimated revenue pipeline (sum of budget ranges for active briefs), top project categories. Use a charting library (Recharts, Chart.js, etc.).

MODULE 03

Performance & Optimization

Database Performance: Proper indexing strategy on frequently queried fields. Use Prisma's `@index` and composite indexes. Cursor-based pagination (not offset). Explain indexing decisions in the README.

Caching Layer: Upstash Redis caching for dashboard analytics (with cache invalidation on state changes) and rate-limiting on the public form and webhook endpoint. Document cache invalidation strategy.

Frontend Optimization: Proper use of React Server Components vs. Client Components. Loading skeletons, suspense boundaries, and optimistic updates on the Kanban. No unnecessary client-side

JavaScript for static content.

API Design: Clean RESTful endpoints with proper HTTP status codes, consistent error response format, and request validation via Zod or similar. No N+1 queries; use Prisma includes/joins deliberately.

TECHNICAL REQUIREMENTS

Stack & Infrastructure

All services below are available on free tiers, zero cost to you.

- Next.js 14+ (App Router)
- PostgreSQL + Prisma
- Neon (free tier)
- Upstash Redis
- Any LLM Provider
- Vercel (free tier)
- Any Auth Solution
- Any UI Library

HOW WE EVALUATE

Evaluation Criteria

CRITERIA	WHAT WE LOOK FOR
Architecture & Schema	Well-normalized data model. Proper relations between briefs, AI analyses, notes,

Design	and events. Clean API layer.
AI Integration Quality	Structured output parsing, graceful failure handling, retry logic. Not just a raw API call and hope for the best.
Performance & Optimization	Intentional indexing, caching with invalidation, cursor pagination, RSC vs. client component decisions, no N+1 queries.
Real-Time & Event-Driven	Live updates work. Optimistic UI on Kanban. Stage transitions logged as event timeline, not just a status field overwrite.
Code Quality & Git	Readable code, error handling beyond happy paths, atomic commits with meaningful messages.
Product Thinking	Works end-to-end. Handles edge cases: AI failure, bad webhook signature, concurrent submissions.
AI-Tool Usage	We expect you to use AI tools. Document what you used and how. We want to see you own and understand the output.

SUBMISSION

What to Submit

1. Deployed app URL (Vercel or equivalent).

2. GitHub repository link (clean README with local setup instructions).
3. Architecture write-up in the README: data model decisions, caching/invalidation strategy, AI pipeline design, what AI tools you used during development, and what you'd improve given more time.

> **A note on effort & fairness:** This assessment is scoped to be completable in 2-3 days of focused work using AI-assisted development workflows. We are not asking for free labor. We are looking for signal that you can deliver a production-quality MVP under realistic constraints. All free-tier services are sufficient. If you're the right person, this is the kind of work you already enjoy doing.

info@codeacme.com · codeacme.com